

วันที่ 6 — วันสุดท้าย

Effective Modelling & Python × MiniZinc

Bounds · Redundant Constraints · Debugging · นำไปใช้จริง

 **Tight Bounds**

solver เร็วขึ้น

 **Redundant**

prune เพิ่ม

 **Debug**

แก้ปัญห ง่ายขึ้น

 **Capstone**

โมเดลจริง

กำหนดการวันที่ 6

09:00–09:45



Variable Bounds

tight bounds ช่วย solver อย่างไร — *before/after* ตัวอย่าง

09:45–10:30



Redundant Constraints

ทำไม *constraint* ที่ 'ซ้ำซ้อน' ช่วยได้ — *lower bound tricks*

10:30–11:00



Debugging Workflow

UNSAT → *Slow* → *Wrong Results* — ขั้นตอนแก้ปัญหา

11:00–12:00



Workshop 6A — Debugging

3 กรณีจริง: *UNSAT*, *Slow*, *Wrong* — หาและแก้ไขปัญหา

13:00–14:30



Dynamic Re-optimization

6 เทคนิครับมือ *plan* เปลี่ยนกลางทาง — รัน 5 ไฟล์ตัวอย่างจริง

14:30–16:00



Python × MiniZinc Capstone

Excel → *MiniZinc* → *Gantt Chart* — รันจริง พร้อมนำเสนอ

Variable Bounds — ช่วย Solver ตัด Space

Solver ทำงานใน search space ที่กำหนดโดย bounds → bounds แน่น = space เล็ก = เร็วกว่า

✗ Bound กว้างเกิน

```
% ✗ Hardcode ค่าสูง
var 0..999: start;
var 0..999: makespan;

% Solver ค้นหาใน 0..999

% = 1,000 ค่า ต่อตัวแปร
```

✓ Bound แน่น

```
% ✓ คำนวณ bound จากข้อมูล
int: total=sum(j,t)(d[j,t]);
int: lb=max(j)(sum(t)(d[j,t]));
var 0..total: start;
var lb..total: makespan;

% lb = lower bound ที่รู้แน่ๆ
```



Rule: lower bound = เร็วที่สุดที่ทฤษฎีบอก | upper bound = ช้าที่สุดที่สมเหตุสมผล

การคำนวณ Bounds ที่ดี

ปัญหา	Lower Bound (lb)	Upper Bound (ub)
Job Shop makespan	$\max(j)(\sum(t)(d[j,t]))$	$\sum(j,t)(d[j,t])$
Bin Packing n_bins	$\text{ceil}(\sum(\text{sizes})/\text{capacity})$	n_items (worst case)
Rostering nights	$\text{ceil}(\text{req_night} * \text{n_days} / \text{n_emp})$	n_days (every night)
VRP total distance	$\sum(\text{min_dist_to_nearest})$	$\sum(\text{all_dist})$ (loose)
Scheduling start	0	$\sum(\text{all_durations}) - \text{min_dur}$

Pattern: `var lb..ub: variable;`

```
int: lb = max(j)(sum(t)(d[j,t])); int: ub = sum(j,t)(d[j,t]);
var lb..ub: makespan; % แทน var 0..999: makespan;
```

Redundant Constraints — เพิ่ม Pruning

Redundant = จริงอยู่แล้วจาก constraint อื่น แต่ช่วย solver ตัดสิ่งที่เป็นไปได้ได้เร็วกว่า

```
% ปัญหา Assignment (แต่ละคนได้งานต่างกัน)
```

```
constraint alldifferent(assigned);
solve minimize sum(w)(cost[w, assigned[w]]);
```

```
% — Redundant constraints เพิ่ม pruning —————
```

```
% Lower bound: ต้นทุนรวมต้องอย่างน้อยเท่ากับ sum ของ min ต่อ worker
```

```
constraint sum(w)(min(t)(cost[w,t])) <= sum(w)(cost[w,assigned[w]]);
```

```
% Lower bound อีกแบบ: sum ของ min ต่อ task
```

```
constraint sum(t)(min(w)(cost[w,t])) <= sum(w)(cost[w,assigned[w]]);
```

ทำไมได้ผล:

Redundant constraint บอก solver

min cost ที่เป็นไปได้ก่อนหาคำตอบ — *prune branches* ที่ *cost* จะเกินได้เลย

ไม่ตัด optimal solution

เพราะ constraint จริงอยู่แล้ว — เป็นแค่ hint ให้ solver ฉลาดขึ้น

Debugging Workflow — 3 ประเภทปัญหา



UNSATIFIABLE

1. เพิ่ม constraint ทีละอย่าง — หว่าอันไหนทำให้ขัดแย้ง
2. ลด scope: เริ่มกับข้อมูลน้อยๆ (2-3 items)
3. คำนวณมือ: ผลลัพธ์ที่เป็นไปได้มีจริงไหม?
4. ผ่อนคลาย constraint แล้วดูว่า UNSAT หายไหม



SLOW / TIMEOUT

1. ตรวจสอบ bounds: กว้างเกินไปไหม? จำนวน lb/ub ที่แน่น
2. เพิ่ม redundant constraints เพื่อช่วย prune
3. เพิ่ม symmetry breaking
4. ลอง solver อื่น: Chuffed, OR-Tools, Gurobi



WRONG RESULT

1. ตรวจสอบ output มือ: จำนวน resource usage แล้วเทียบ
2. ลอง solve satisfy ก่อน ดูว่า model ถูก
3. เพิ่ม assert() ตรวจสอบ data ที่รับมา
4. ตรวจสอบ array index: แกว/คอลัมน์สลับกันไหม?



Workshop 6A — Debugging Challenge

เวลา: 60 นาที | ไฟล์: `debug_broken_1.mzn`, `debug_broken_2.mzn`, `debug_broken_3.mzn`



กรณี 1: UNSATISFIABLE (`debug_broken_1.mzn`)

Maintenance Scheduling — deadline ที่กำหนดเป็นไปไม่ได้

ภารกิจ: หาว่า deadline ต้องเป็นเท่าไรถึงจะ satisfy? คำนวณมือก่อนแล้วยืนยันด้วย MiniZinc



กรณี 2: ช้ามาก (`debug_broken_2.mzn`)

Job Shop 6x4 — bounds hardcoded เป็น 200 ทั้งที่คำนวณได้

ภารกิจ: คำนวณ total และ lb ที่แน่น แล้วเปรียบเทียบ solve time



กรณี 3: ผลไม่ถูกต้อง (`debug_broken_3.mzn`)

Production Planning — model รันได้แต่ resource usage เกิน capacity

ภารกิจ: หา bug แล้วแก้ไข ยืนยันด้วยการคำนวณมือ

Dynamic Re-optimization

รับมือเมื่อแผน เปลี่ยนกลางทาง

เครื่องเสีย · Order เพิ่ม · วัตถุดิบขาด · พนักงานลา

ทำไม Plan ถึงเปลี่ยนกลางทาง?

เครื่องจักรขัดข้อง

M_B เสียกะทันหัน
งาน Weld ทั้งหมดต้องย้ายเครื่อง
Makespan จะบวมขึ้นเท่าไร?

Order เพิ่มกะทันหัน

ลูกค้า VIP ส่ง order ใหม่
ความต้องการ +40% กะทันหัน
Capacity พอไหม? OT ต้องเท่าไร?

วัตถุดิบมาช้า / ขาด

Supplier แจ้งว่าส่งช้า 3 วัน
งานที่รอวัตถุดิบต้องเลื่อน
Ripple effect ทั้ง schedule

คำถาม: แก่ plan อย่างไร? — Re-plan ใหม่ทั้งหมด vs. แก้น้อยที่สุด vs. วางแผนรับมือล่วงหน้า

day6/ 06_reoptimize.mzn | 07_penalize_changes.mzn | 08_rolling_horizon.mzn |
09_slack_variables.mzn | 10_scenario_planning.mzn

6 เทคนิครับมือการเปลี่ยนแปลง

1. Re-optimize

อัปเดตข้อมูล แล้ว solve ใหม่

2. Penalize Changes

ลงโทษการเบี่ยงเบนจาก plan เดิม

3. Rolling Horizon

Solve ที่ละ window แล้วเลื่อนไป

4. Slack Variables

Overtime buffer สำหรับ demand spike

5. Scenario Planning

Optimize under multiple demand scenarios

6. Warm Starting

ใช้ solution เดิมเป็น hint แก่ solver

1. Re-optimize from Current State

แนวคิด

เมื่อ event เกิดขึ้น:

1. อัปเดตข้อมูลให้ตรงสถานการณ์
2. Fix งานที่เริ่มไปแล้ว (start time)
3. Run solver ใหม่ด้วยข้อมูลใหม่

ได้ plan ที่ optimal สำหรับสถานการณ์ใหม่

แต่ plan อาจเปลี่ยนมาก → shop floor สับสน

```
%% 06_reoptimize.mzn
```

```
%% แก้เปลี่ยน 1 บรรทัด แล้ว re-run!
```

```
bool: M_B_available = false;
```

```
array[TASK,MACHINE] of bool: capable =
  array2d(TASK, MACHINE, [
    true,  M_B_available, true,  %% Cut
    false, M_B_available, true,  %% Weld
    true,  M_B_available, true,  %% Finish
  ]);
```

```
solve minimize makespan;
```

เหมาะกะ: เหตุการณ์ไม่คาดฝัน ที่ต้องการ plan ใหม่ทั้งหมด | ข้อเสีย: plan เปลี่ยนมาก สื่อสารยาก

2. Penalize Changing the Plan

Objective = makespan + alpha × disruption

disruption = จำนวน task ที่ถูกย้ายเครื่องจาก plan เดิม

ปรับ alpha

alpha = 0 → ไม่สนใจ plan เดิม

alpha = 5 → สมดุล speed vs. stability

alpha = 50 → รักษา plan เดิมเป็นหลัก

```
int: alpha = 5;
```

```
var 0..N: disruption =  
    sum(j,t) (bool2int(  
        machine[j,t] != orig_machine[j,t]  
    ));
```

```
solve minimize  
    makespan + alpha * disruption;
```

→ ลองรัน: day6/07_penalize_changes.mzn | ทดลองเปลี่ยน alpha = 0, 5, 50 แล้วเปรียบเทียบ

3. Rolling Horizon Optimization

Window 1: D1-D3 ✓ solved, fixed

Window 2: D4-D6 ← solving now

Window 3: D7-D9 (unknown)

ข้อดี

ปัญหาเล็กลง → solve เร็วกว่ามาก

รับ demand จริงทุก window (ปรับได้ตาม reality)

ไม่ต้อง commit กับ plan ไกล ๆ ที่ไม่แน่นอน

วิธีรัน 2 windows

horizon_start = 1 → solve D1-D3

บันทึก stock ปลาย window

horizon_start = 4 → solve D4-D6

ใส่ initial_stock = [3, 0, 2]

→ day6/08_rolling_horizon.mzn | เปลี่ยน horizon_start = 1 แล้ว 4 ดูผลต่างกัน

4. Slack Variables — Overtime Buffer

แนวคิด

เพิ่ม overtime เป็น "slack buffer"

$\text{regular} + \text{overtime} \geq \text{demand}$

$\text{regular_cap} = 40\text{h}$ (ปกติ)

$\text{max_overtime} = 12\text{h}$ (กรณีฉุกเฉิน)

Objective: minimize total_cost
= regular_cost * regular
+ overtime_cost * overtime

```
%% 09_slack_variables.mzn
```

```
var 0..100: regular[m,p];  
var 0..20:  overtime[m,p];
```

```
constraint forall(m,p) (  
    regular[m,p]  <= normal_cap[m] /\  
    overtime[m,p] <= max_overtime[m] /\  
    regular[m,p] + overtime[m,p]  
    >= hours_needed[m,p]  
);
```

```
solve minimize total_cost;
```

เหมาะกับ: demand spike ที่รู้ล่วงหน้า / ตอบได้ว่า "รับ order ได้ไหม? ต้นทุน OT เท่าไหร่?"

5. Scenario-based Planning

S1 Normal (50%)

demand ปกติ

S2 Surge (30%)

demand +40%

S3 Decline (20%)

demand -25%



แผนเดียวที่ **minimize expected cost** ทุก scenario

$\min \sum \text{prob}[s] \times \text{cost}(\text{plan}, s)$ s.t. แผนต้องใช้อีก่อนรู้ scenario

```
var int: expected_cost =  
  sum(s in SCENARIO) (  
    prob[s] * (unit_cost*produce + holding*leftover + stockout*unmet)  
  ) div 100;  
solve minimize expected_cost;
```

→ day6/10_scenario_planning.mzn | ลอง: ลด stockout_cost / เพิ่ม prob[Surge] แล้วดูว่า plan เปลี่ยนยังไง

6. Warm Starting — ให้ Hint แก่ Solver

แนวคิด

ให้ solution เดิมเป็น "initial hint" แก่ solver

→ Solver เริ่มจากจุดที่ดี ไม่ต้อง explore จาก scratch

มีประโยชน์มากเมื่อ:

- ปัญหาใหญ่ / solve นาน
- Solve ซ้ำบ่อย (real-time re-planning)
- Plan เปลี่ยนนิดเดียว (เช่น เพิ่ม 1 job)

Gecode (default solver): ใช้ constraint fix บางส่วนเป็น hint

OR-Tools / Gurobi / CPLEX มี API-level warm start ที่ทรงพลังกว่า

```
%% Warm start: fix known part of solution
%% ด้วย constraint ขั้วครว
```

```
constraint machine[OrderA, Cut] = M_A;
constraint machine[OrderA, Weld] = M_C;
constraint start[OrderA, Cut] = 0;
```

```
%% หรือใช้ search annotation:
```

```
solve
:: int_search([machine[j,t]|j,t],
              first fail, indomain median)
```


สรุป: เลือก Technique ไหน?

สถานการณ์	Technique	ไฟล์ตัวอย่าง
เครื่องเสีย / demand spike จุกเงิน	1. Re-optimize	06_reoptimize.mzn
ต้องการ plan เปลี่ยนน้อยที่สุด	2. Penalize Changes	07_penalize_changes.mzn
Horizon ยาว / demand ไม่แน่นอนไกล	3. Rolling Horizon	08_rolling_horizon.mzn
รับ order พิเศษ / overtime ได้	4. Slack Variables	09_slack_variables.mzn
ตัดสินใจล่วงหน้าภายใต้ uncertainty	5. Scenario Planning	10_scenario_planning.mzn
ปัญหาใหญ่ / solve ช้าบ่อย	6. Warm Starting	(เพิ่ม constraint hint)

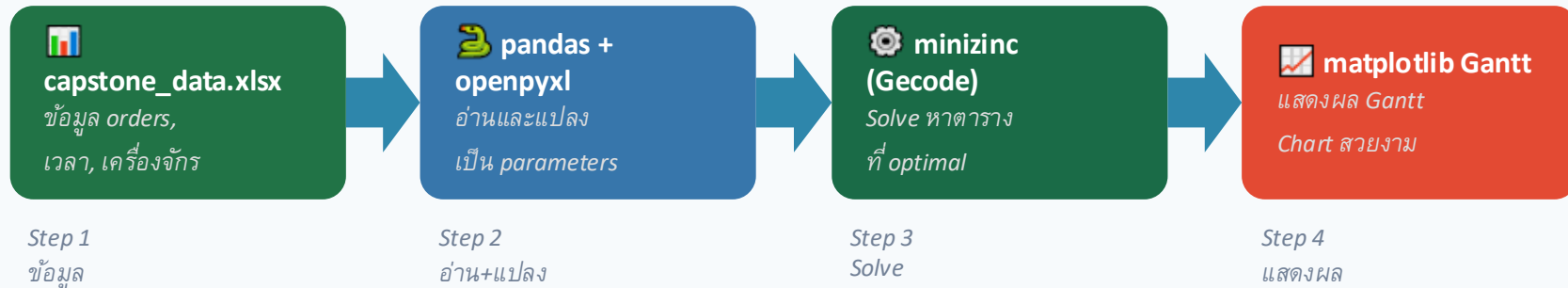
💡 ในทางปฏิบัติ: ผสมหลาย technique เช่น **Rolling Horizon + Penalize Changes + Slack Variables**

Capstone — Integration Workshop

Python × MiniZinc × Excel × Matplotlib

อ่านข้อมูลจาก Excel → Solve ด้วย MiniZinc → แสดงผล Gantt Chart

ทำไมต้องใช้ Python ร่วมกับ MiniZinc?



ไฟล์ในโฟลเดอร์ day6/capstone/

jobshop_model.mzn | capstone_data.xlsx | run_capstone.py



Integration

ต่อกับ ERP, Excel, Database ขององค์กรได้



Automation

รัน schedule ใหม่อัตโนมัติทุกวันหรือทุก shift



Visualization

แสดงผลสวยงาม ผู้บริหารเข้าใจง่าย

Code: อ่าน Excel → ส่งให้ MiniZinc

STEP 1-2: อ่าน Excel + เตรียม parameters

```
import pandas as pd
import minizinc

# อ่านข้อมูลจาก Excel
df_j = pd.read_excel('capstone_data.xlsx',
                    sheet_name='orders')
df_p = pd.read_excel('capstone_data.xlsx',
                    sheet_name='proc_times')
```

```
n_job, n_task, n_machine = 4, 3, 3
```

```
# สร้าง dur[j,t,m] — flatten เป็น list
```

```
pip install minizinc pandas openpyxl matplotlib
```

MiniZinc ต้องติดตั้งแยก: <https://www.minizinc.org/software.html>

```
for t in range(1, n_task+1):
    row = df_p[(df_p.JobID==j) &
              (df_p.TaskID==t)]
    for m in ['M_A', 'M_B', 'M_C']:
        dur_flat.append(int(row[m]))
```

STEP 3: รัน MiniZinc via Python API

```
# โหลด model และ solver
model = minizinc.Model(
    ['jobshop_model.mzn'])
solver = minizinc.Solver.lookup(
    "gecode")
inst = minizinc.Instance(
    solver, model)
```

```
# ส่ง parameters
```

```
inst["n_job"]      = n_job
inst["n_task"]     = n_task
inst["n_machine"]  = n_machine
inst["dur"]        = dur_flat
```

```
result = inst.solve()
makespan = result["makespan"]
start    = result["start"]
machine  = result["machine"]
```

Code: แสดง Gantt Chart + ทดลองรัน

STEP 4: Gantt Chart ด้วย matplotlib

```
import matplotlib.pyplot as plt
import matplotlib.patches as mp
```

```
COLORS = ["#2E86AB", "#D97706",
          "#1A6B47", "#DC2626"]
```

```
fig, ax = plt.subplots()
for j in range(n_job):
    for t in range(n_task):
        m = machine[j][t] - 1
        d = dur[j][t][m]
        ax.barh(
            y=m, width=d,
            left=start[j][t],
            color=COLORS[j])
```

Gantt Chart Output (schedule_result.png)



🚀 Hands-on: รันเลย!

```
cd day6/capstone/
```

```
python run_capstone.py
```

ลองแก้ capstone_data.xlsx → เพิ่ม order ใหม่ → รันใหม่ → Gantt chart เปลี่ยนยังไง?

สรุปหลักสูตร 6 วัน

1

วันที่ 1: พื้นฐาน

var, constraint, solve satisfy/minimize/maximize, data files

2

วันที่ 2: Arrays & Enums

forall, sum, array indexing, enum types, bool2int

3

วันที่ 3: Global Constraints

alldifferent, cumulative, table, disjunctive, Job Shop

4

วันที่ 4: Scheduling

predicate, Flexible Job Shop, Rostering, regular

5

วันที่ 5: Routing & Packing

circuit/TSP/VRP, bin_packing_capa, Knapsack

6

วันที่ 6: Effective Modelling

tight bounds, redundant constraints, debugging, Capstone

Quick Reference: Global Constraints ทั้งหมด

Constraint	Include	ใช้กับ	ตัวอย่าง
<code>alldifferent(x)</code>	<code>globals.mzn</code>	Assignment, no-dup	<code>alldifferent(assigned)</code>
<code>cumulative(s,d,r,b)</code>	<code>cumulative.mzn</code>	Resource scheduling	<code>cumulative(start,dur,workers,cap)</code>
<code>disjunctive(s,d)</code>	<code>disjunctive.mzn</code>	Machine no-overlap	<code>disjunctive([s[j],t] j in J],[d[j],t] j in J)</code>
<code>table(x,tuples)</code>	<code>table.mzn</code>	Lookup/spec table	<code>table([mat,hard,wt,cost],mat_table)</code>
<code>regular(seq,tr,q0,F)</code>	<code>regular.mzn</code>	Shift patterns	<code>regular([roster[e,d] d in D],tr,S1,STATE)</code>
<code>circuit(next)</code>	<code>circuit.mzn</code>	TSP/VRP routing	<code>circuit(next)</code>
<code>bin_packing_capa(c,b,w)</code>	<code>bin_packing_capa.mzn</code>	Bin packing	<code>bin_packing_capa(cap,bin_of,sizes)</code>

Checklist ก่อน Submit โมเดล

☐ Bounds แน่น

ไม่ใช่ 0..9999 — คำนวณ lb/ub จากข้อมูลจริง

☐ ข้อมูลขนาดเล็กก่อน

ทดสอบกับ $n=2,3$ ก่อน scale ขึ้น

☐ มี `assert()` ตรวจสอบ data

บอก error ที่ชัดเจนถ้า data ผิด

☒ solve satisfy ก่อน

ทดสอบว่า model ถูกก่อนเพิ่ม objective

☐ ตรวจสอบ output มือ

คำนวณ resource usage เทียบกับ constraint

☐ Comment อธิบาย constraint

คนอื่น (หรือตัวเองในอีก 1 เดือน) อ่านรู้เรื่อง

ขั้นต่อไปหลังอบรม

แหล่งเรียนรู้เพิ่มเติม

- ▶ docs.minizinc.dev — Official tutorial
- ▶ minizinc.org/challenge — ตัวอย่างโมเดลจาก competition
- ▶ MiniZinc Playground — ทดลองออนไลน์
- ▶ Python + minizinc package — เชื่อม automation
- ▶ Gurobi/CPLEX — สำหรับปัญหาใหญ่ระดับ enterprise

นำไปใช้จริง

- ✓ นำ Capstone model ไปทดสอบกับข้อมูลจริง
- ✓ เพิ่ม solver แข่งพาดิษฐ์ถ้าปัญหาใหญ่ขึ้น
- ✓ ขยาย model เพิ่ม constraint ใหม่ตามความต้องการ
- ✓ แชร model กับทีม — ทำ documentation
- ✓ ติดต่อวิทยากรถ้าติดปัญหา



ขอแสดงความยินดี!

ท่านสามารถใช้ MiniZinc แก้ปัญหา Optimization
ในงานอุตสาหกรรมได้แล้ว

Variables

Constraints

forall/sum

Arrays

Enum

Global Constraints

cumulative

alldifferent

Job Shop

Rostering

regular

VRP

Bin Packing

Knapsack

Debugging

Quick Reference: Effective Modelling

```
% — TIGHT BOUNDS —————

int: total = sum(j,t)(d[j,t]);      % upper bound
int: lb     = max(j)(sum(t)(d[j,t])); % lower bound
var lb..total: makespan;

% — REDUNDANT CONSTRAINTS —————

% Lower bound: objective >= sum(row_min)
constraint sum(w)(min(t)(cost[w,t])) <= sum(w)(cost[w,assigned[w]]);

% — SYMMETRY BREAKING (examples) —————

% Bin Packing: bin_of[i] <= bin_of[i+1]+1
constraint forall(i in 1..n-1)(bin_of[i] <= bin_of[i+1]+1);

% Job Shop: ถ้า 2 jobs ทำ task เดียวกันพร้อมกัน force j<k
% Assignment: worker 1 ใ้ task หมายเลขน้อยที่สุด
constraint assigned[1] = min(TASKS); % อาจเร็วขึ้น
```

Quick Reference: Debugging

% — UNSATISFIABLE —

% 1. เพิ่ม constraint ที่ละอย่าง: เริ่ม solve satisfy ก่อน

`solve satisfy;` % ไม่ minimize ก่อน

% 2. ลด n: เริ่มด้วยข้อมูล 2-3 items

% 3. Comment constraint ออก แล้วเพิ่มทีละอย่าง

% — SLOW SOLVER —

% 1. กำหนด bounds แทน hardcoded

`int: ub = sum(j,t)(d[j,t]);` % แทน var 0..999

% 2. เพิ่ม symmetry breaking

% 3. ลอง solver อื่น: `--solver chuffed` หรือ `or-tools`

% — WRONG RESULTS —

% 1. เพิ่ม `assert()` ตรวจสอบ input data

`constraint assert(n > 0, "n must be positive");`

% 2. เพิ่ม output ตรวจสอบ intermediate values

วันที่ 6: Effective Modelling - Quick Ref: Debugging `min(sum(p)(c[p,r]*x[p])), "\n");`

ตัดสินใจเลือก Pattern ให้ถูก

ต้องการ	Pattern	วันที่เรียน
ทุกค่าต้องต่างกัน	<code>alldifferent(x)</code>	วัน 3
ทรัพยากรไม่เกิน <code>cap</code> ทุกเวลา	<code>cumulative(s,d,r,cap)</code>	วัน 3
เครื่องทีละ 1 job	<code>disjunctive(s,d)</code>	วัน 3
เลือกจาก tuple ที่กำหนด	<code>table(x, allowed)</code>	วัน 3
บังคับ shift pattern	<code>regular(seq,tr,q0,F)</code>	วัน 4
Hamiltonian cycle (TSP/VRP)	<code>circuit(next)</code>	วัน 5
จัดสิ่งของลงกล่อง	<code>bin_packing_capa(cap,b,w)</code>	วัน 5
เลือก item maximize profit	0/1 Knapsack หรือ var set	วัน 5
ลด duplicate solutions	Symmetry breaking constraint	วัน 5/6
เพิ่ม pruning speed	Redundant constraints	วัน 6

ข้อผิดพลาดที่พบบ่อย — Effective Modelling

❌ `var 0..1000: makespan % hardcoded bound`

✅ `int: ub = sum(j,t)(d[j,t]); var lb..ub: makespan;`

Bound ที่ดีคำนวณได้เสมอ — ไม่ต้อง guess

❌ `solve minimize cost; % ก่อนทดสอบ satisfy`

✅ `solve satisfy; % ทดสอบก่อน, เปลี่ยนเป็น minimize ที่หลัง`

เริ่ม *minimize* โดยไม่ตรวจว่า *model* ถูก → UNSAT หาสาเหตุยาก

❌ `% ไม่มี comment อธิบาย constraint`

✅ `% Demand: กะเข้าต้องมีพนักงานอย่างน้อย req_morning คน`

Model ที่ไม่มี *comment* อ่านยากมาก โดยเฉพาะเมื่อกลับมาแก้ทีหลัง

❌ `forall(i,j in 1..n where i<j)(sym_break[i,j])`

✅ `% ตรวจว่า symmetry breaking ไม่ตัด optimal solution ออก`

Symmetry breaking ผิดอาจตัดคำตอบที่ดีออก — ต้องระวัง

Solver ที่รองรับใน MiniZinc

Solver	ประเภท	เหมาะกับ	Flag
Gecode	CP (default)	Integer, Scheduling, Rostering	<code>--solver gecode</code>
Chuffed	CP (lazy)	Scheduling ใหญ่	<code>--solver chuffed</code>
OR-Tools	CP+MIP	Mixed problems	<code>--solver org.minizinc.ortools</code>
HiGHS	MIP (free)	Linear/Integer Programming	<code>--solver highs</code>
COIN-BC	MIP (free)	Linear Programming	<code>--solver cbc</code>
Gurobi	MIP (commercial)	Production LP/MIP ขนาดใหญ่	<code>--solver gurobi</code>

แนะนำสำหรับผู้เริ่มต้น:

Gecode (default) สำหรับปัญหาทั่วไป | Chuffed สำหรับ Scheduling | HiGHS/COIN-BC สำหรับปัญหา Linear

สรุปวันที่ 6 — Effective Modelling & Dynamic Re-optimization



Tight Bounds — solver เร็วขึ้น

lb = min possible | ub = max reasonable | var lb..ub แทน var 0..999



Redundant Constraints — prune เพิ่ม

constraint ที่ซ้ำซ้อนทางตรรกะ แต่ช่วย solver ตัด branch ได้เร็ว



Debugging: 3 ประเภท

UNSAT → เพิ่ม constraint ที่ละเอียด | Slow → bounds + redundant | Wrong → ตรวจสอบ output มือ



Dynamic Re-optimization — 6 เทคนิค

Re-opt / Penalize / Rolling Horizon / Slack / Scenario Planning / Warm Start



Capstone = นำไปใช้จริง

ปัญหาจริงขององค์กร → MiniZinc model → ผลลัพธ์ที่ดีขึ้น



ยินดีด้วย! ท่านจบหลักสูตร MiniZinc for Industrial Optimization แล้ว